

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Flight Data Analysis with Apache Spark

Batch analysis of historical data

Nicola Violante (0362469)

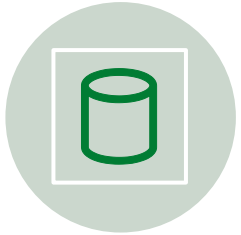
Agenda



INTRODUCTION &
PROJECT OBJECTIVES



SYSTEM
ARCHITECTURE



DATA INGESTION
PIPELINE & STORAGE
VOLUME ANALYSIS



SPARK APPLICATION
DESIGN



QUERY
IMPLEMENTATION



SERVING LAYER &
VISUALIZATION



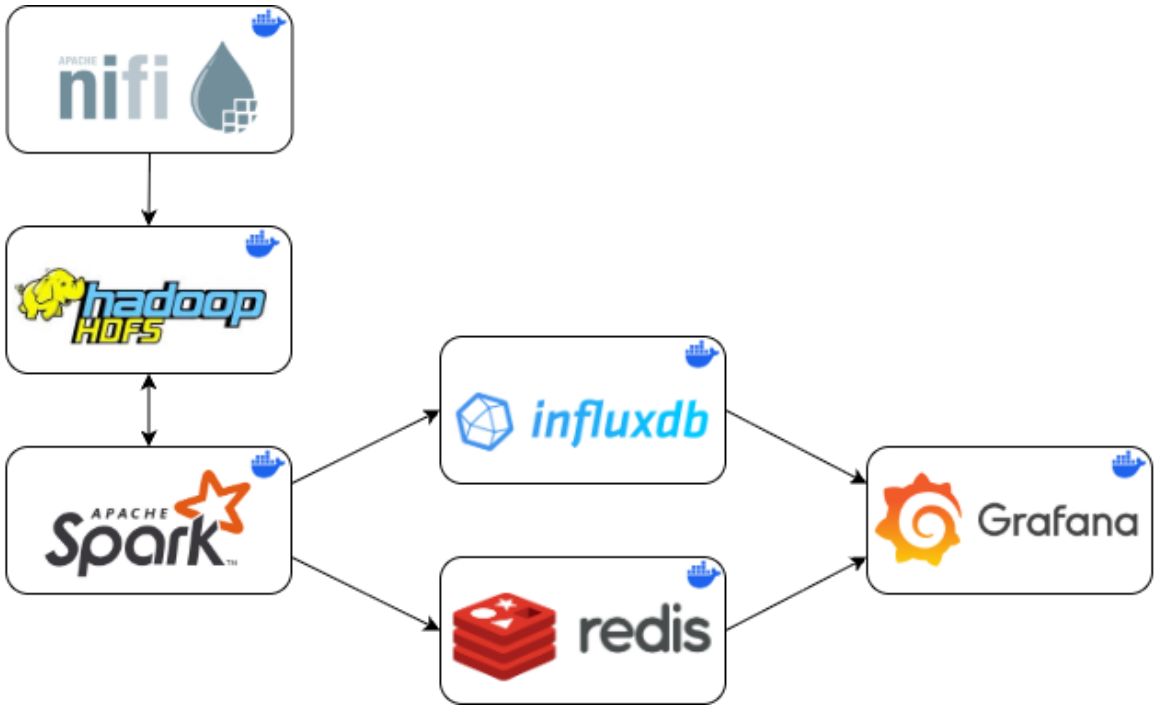
PERFORMANCE
BENCHMARKING

Introduction & Project Objectives

- Context: Batch analysis of historical flight data provided by the United States department of Transportation
- Dataset dimensions: Preprocessed dataset of 2.200.000 events (January 1st - April 30th, 2025)
- Objectives:
  - Implementation of two analytical queries
  - Performance evaluation
- Parameters evaluated:
  - Spark APIs (RDD, Dataframe, SQL)
  - Data storage format (CSV vs Parquet)
  - Horizontal parallelism degrees

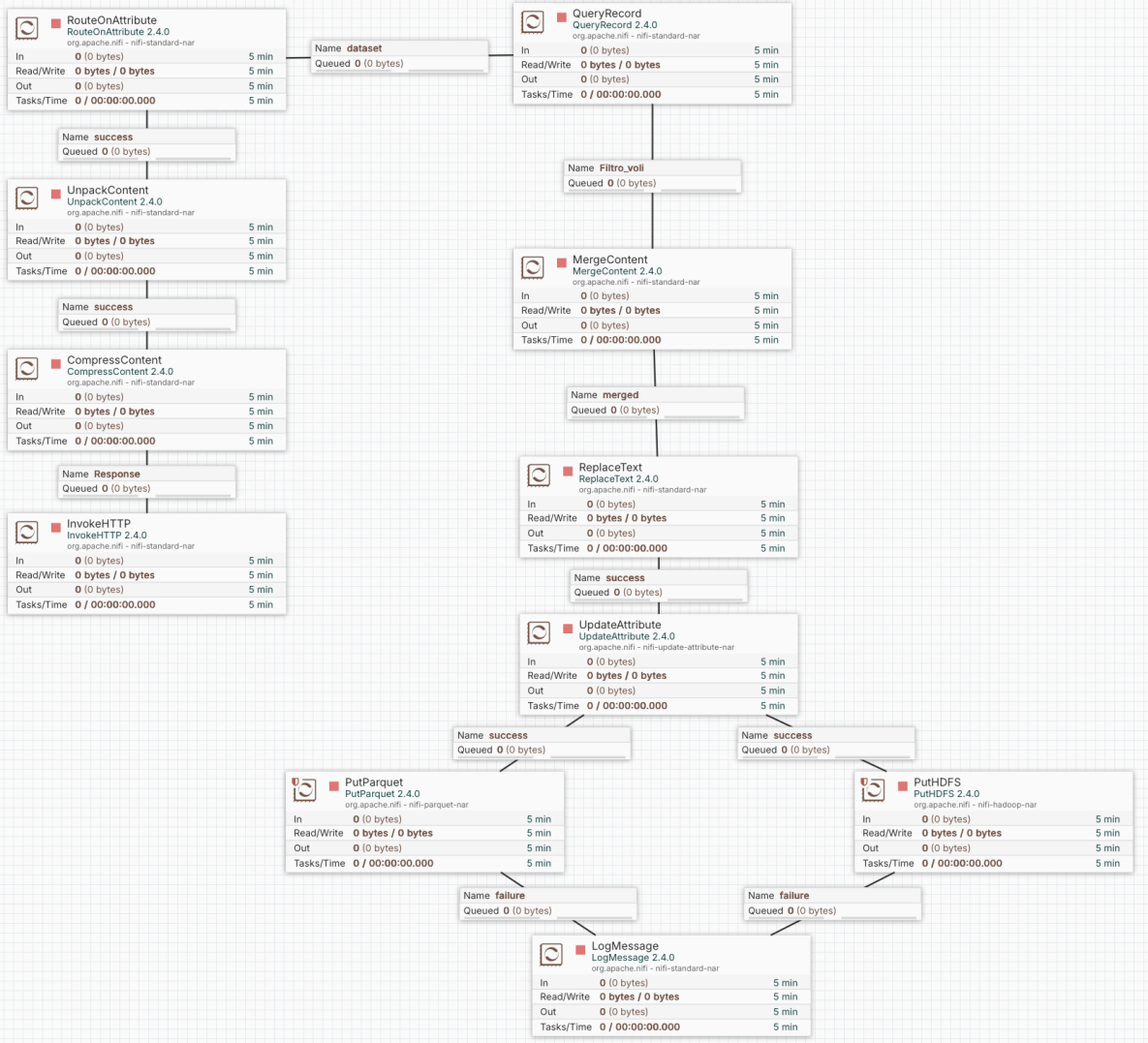


System Architecture



- Ecosystem orchestrated via **Docker Compose**
- Ingestion Layer: **Apache Nifi**
- Storage Layer: **Hadoop HDFS**
- Processing Layer: **Apache Spark**
- Serving Layer: **InfluxDB & Redis**
- Visualization Layer: **Grafana**

Data Ingestion Pipeline & Storage Volume Analysis



- Orchestrated via REST APIs + shell script
- ETL paradigm
- Designed to overwrite old dataset on HDFS
- Storage Volume Analysis:
  - Initial compressed remote archive: **51.75 MB**
  - Uncompressed data: **244.17 MB**
  - Post column filtering and aggregation: **77.18 MB**
  - Parquet format: **9.01 MB**

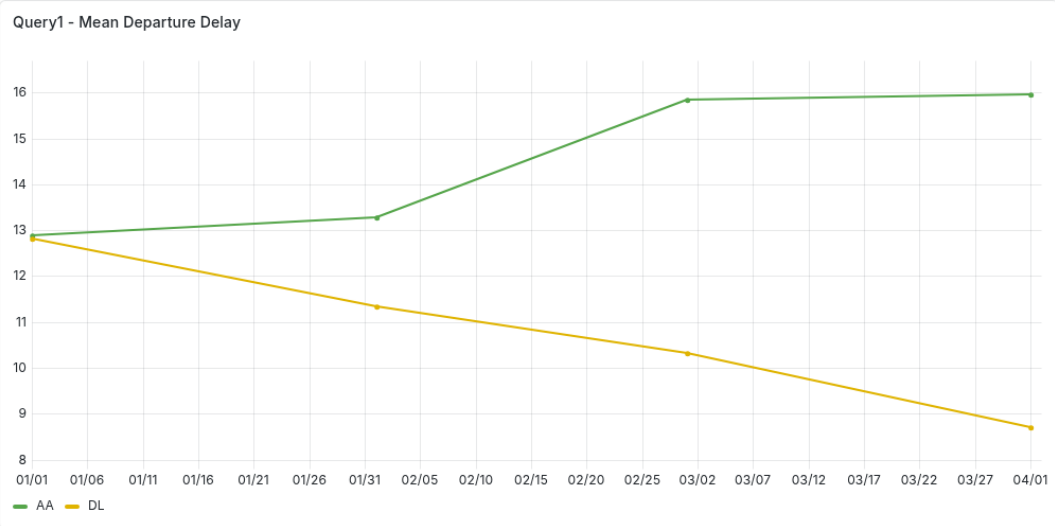
Spark Application Design

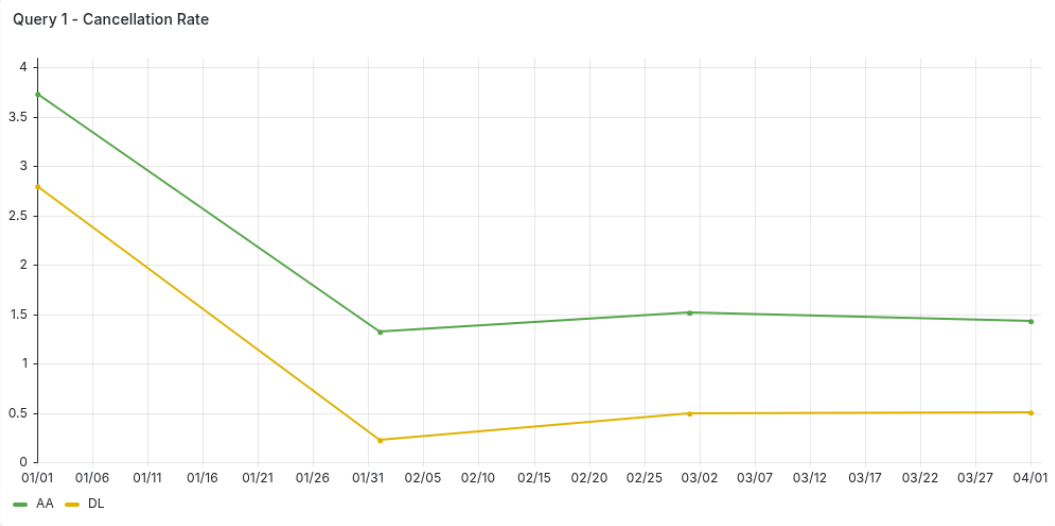
- Dynamic routing
  - Centralized entry points
  - Command line arguments + importlib library
- Separation of concerns
  - Decoupling between I/O operations and analytical business
- Utility abstraction
  - Helper modules
- Algorithmic isolation
  - Benchmark evaluates only Spark's engine efficiency





Query 1 – Monthly delay and cancellation trend



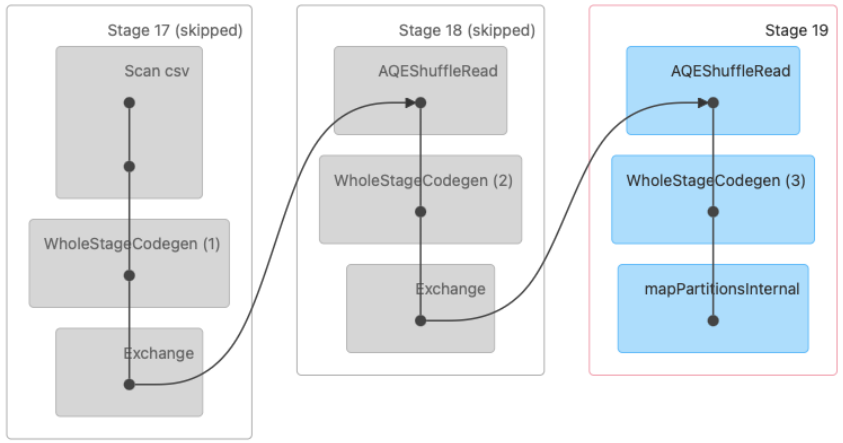


- Objectives:
  - Compute monthly min, max and average departure delay
  - Compute cancellation rate
- Target: American Airlines, Delta
- Constraint:
  - Exclude canceled flights from delay statistics

Query 1 - Implementation

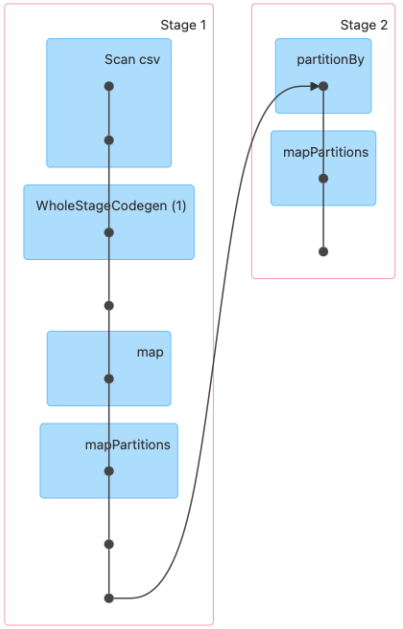
Declarative approach (DataFrame & SQL)

- Pre-aggregation filtering and pushdown
- Conditional aggregation
- Sorting

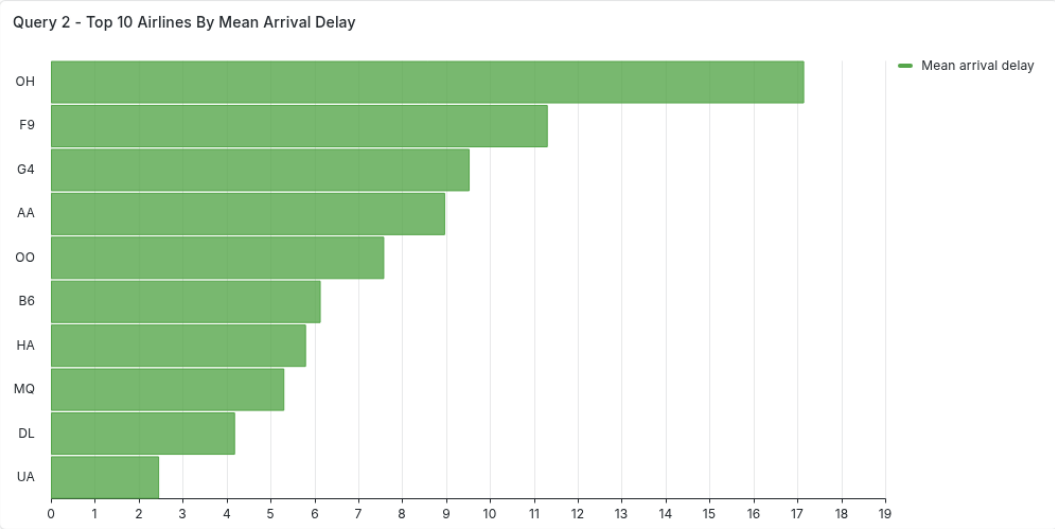


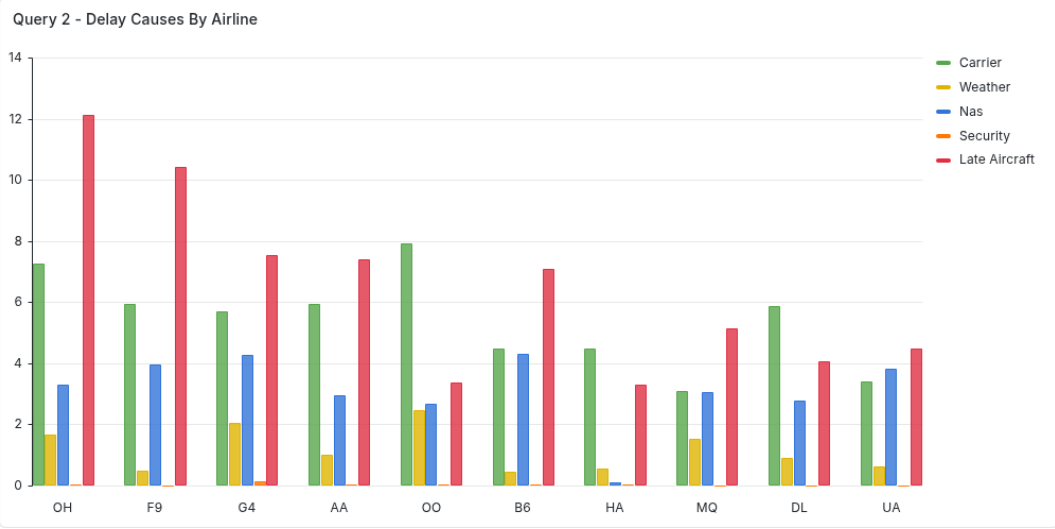
Imperative approach (RDD)

- Row-parsing and map-side filtering
- Map-side combine
- Finalization



Query 2 – Carrier ranking and delay causes



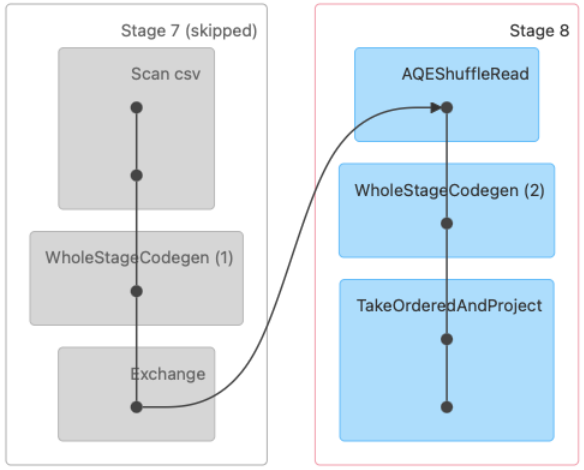


- Objectives:
  - Compute valid flights (non cancelled or diverted)
  - Compute average arrival delay and delay causes
  - Ranking carriers by average arrival delay
- Target: Top 10 airlines with 500+ valid flights
- Methodological imputation:
  - Missing values treated as zero

Query 2 - Implementation

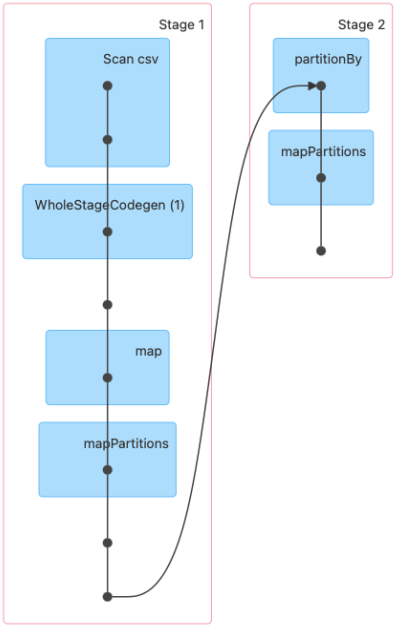
Declarative approach (DataFrame & SQL)

- Pre-aggregation filtering and pushdown
- Global aggregation
- Post-aggregation filtering and ranking



Imperative approach (RDD)

- Row-parsing and map-side filtering
- Map-side combine
- Manual averaging and bounded ranking



Serving Layer & Visualization

- InfluxDB → Query 1 & Benchmark
  - Query 1:
    - Timestamp = First day of month
    - Tag = Carrier code
    - Fields = Analytical results
  - Benchmark:
    - Tag = # of workers
    - Fields = Execution time & Run number
- Redis → Query 2
  - Array of dictionaries serialized into JSON
- Grafana
  - Query 1 → Flux language (data filtered by measurement and field)
  - Query 2 → GET command (extractFields → transponse → extractFields → filterFieldsByName)

Performance Benchmarking

- Implementation:
  - Cold start mitigation
  - Caching elimination
  - Pure algorithmic benchmarking

- Evaluations:
  - Performance of APIs
  - Storage format implications
  - Horizontal scalability
  - Hardware asymmetry

QUERY 1 EXECUTION PERFORMANCE METRICS

API & Format	Workers	Min (s)	Max (s)	Mean (s)	Median (s)	Variance
DataFrame CSV	1	2.66	3.71	2.83	2.72	0.0905
	2	3.55	4.71	3.79	3.68	0.111
	3	4.13	4.99	4.32	4.25	0.0570
	4	4.97	5.67	5.15	5.09	0.0362
DataFrame Parquet	1	2.37	3.14	2.52	2.44	0.0454
	2	3.16	3.78	3.39	3.38	0.0256
	3	3.82	4.47	3.97	3.94	0.0312
	4	4.54	5.08	4.76	4.75	0.0251
RDD CSV	1	4.59	6.69	4.94	4.73	0.346
	2	5.23	6.49	5.51	5.41	0.121
	3	5.74	6.87	6.08	5.93	0.117
	4	6.65	7.54	6.99	6.99	0.0463
RDD Parquet	1	6.30	7.44	6.52	6.40	0.108
	2	7.03	7.79	7.35	7.38	0.0504
	3	7.38	8.16	7.73	7.79	0.0609
	4	8.18	8.85	8.54	8.58	0.0575
SQL CSV	1	2.61	3.60	2.78	2.70	0.0766
	2	3.51	4.52	3.75	3.67	0.0755
	3	4.13	5.00	4.32	4.27	0.0538
	4	4.94	5.85	5.17	5.13	0.0568
SQL Parquet	1	2.36	3.19	2.51	2.44	0.0545
	2	3.20	3.95	3.37	3.32	0.0444
	3	3.66	4.48	3.95	3.92	0.0548
	4	4.36	5.26	4.71	4.67	0.0508

QUERY 2 EXECUTION PERFORMANCE METRICS

API & Format	Workers	Min (s)	Max (s)	Mean (s)	Median (s)	Variance
DataFrame CSV	1	2.92	3.97	3.08	2.97	0.0913
	2	3.74	4.66	3.96	3.89	0.0579
	3	4.28	5.08	4.42	4.35	0.0528
	4	5.51	6.77	5.81	5.66	0.123
DataFrame Parquet	1	2.70	3.38	2.81	2.73	0.0383
	2	3.27	4.33	3.59	3.56	0.0982
	3	3.80	4.37	4.19	4.26	0.0311
	4	4.44	5.31	4.98	4.98	0.0614
RDD CSV	1	6.17	7.08	6.36	6.28	0.0613
	2	6.04	6.91	6.33	6.23	0.0641
	3	6.44	7.05	6.58	6.52	0.0315
	4	7.19	8.05	7.52	7.43	0.0639
RDD Parquet	1	9.64	10.30	9.85	9.83	0.0285
	2	10.20	11.70	10.70	10.60	0.134
	3	10.70	11.70	11.10	11.00	0.113
	4	12.40	12.70	12.50	12.50	0.00704
SQL CSV	1	2.90	3.86	3.08	2.99	0.0696
	2	3.77	4.78	3.91	3.81	0.0853
	3	4.25	5.12	4.47	4.35	0.0825
	4	5.53	6.28	5.73	5.67	0.0419
SQL Parquet	1	2.69	3.36	2.81	2.74	0.0355
	2	3.25	4.35	3.54	3.38	0.114
	3	3.76	4.41	4.06	4.08	0.0483
	4	4.48	5.38	4.95	4.98	0.0709



TOR VERGATA

UNIVERSITÀ DEGLI STUDI DI ROMA